Overlay of Maps

1. Output Sensitive Algorithms
   - the line segment intersection problem
   - handling event points
2. Correctness and Cost Analysis
   - correctness
   - cost analysis
3. Doubly Connected Edge Lists
   - representing planar subdivisions
   - vertex, face, and edge records

MCS 481 Lecture 5
Computational Geometry
Jan Verschelde, 2 September 2026

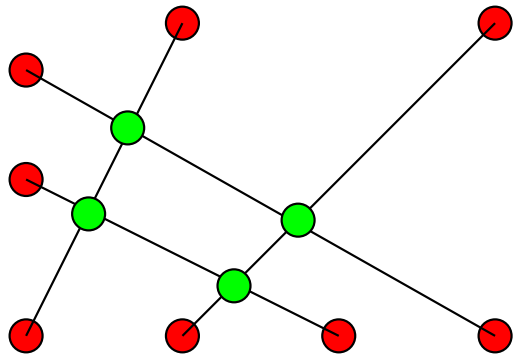
Overlay of Maps

1. Output Sensitive Algorithms
   - the line segment intersection problem
   - handling event points
2. Correctness and Cost Analysis
   - correctness
   - cost analysis
3. Doubly Connected Edge Lists
   - representing planar subdivisions
   - vertex, face, and edge records

the line segment intersection problem

Input: n line segments $S = \{s_1, s_2, \ldots, s_n\}$, $\#S = n < \infty$,
$s_i = ((a_x^i, a_y^i), (b_x^i, b_y^i))$, $i = 1, 2, \ldots, n$.

Output: intersection points $P = \{\ ((i,j),(x,y)) \mid (x,y) \in s_i \cap s_j\ \}$.



The worst case complexity is $\Omega(n^2)$.

The running time of an *output sensitive algorithm*
is proportional to the size of the output.

the algorithm FINDINTERSECTIONS

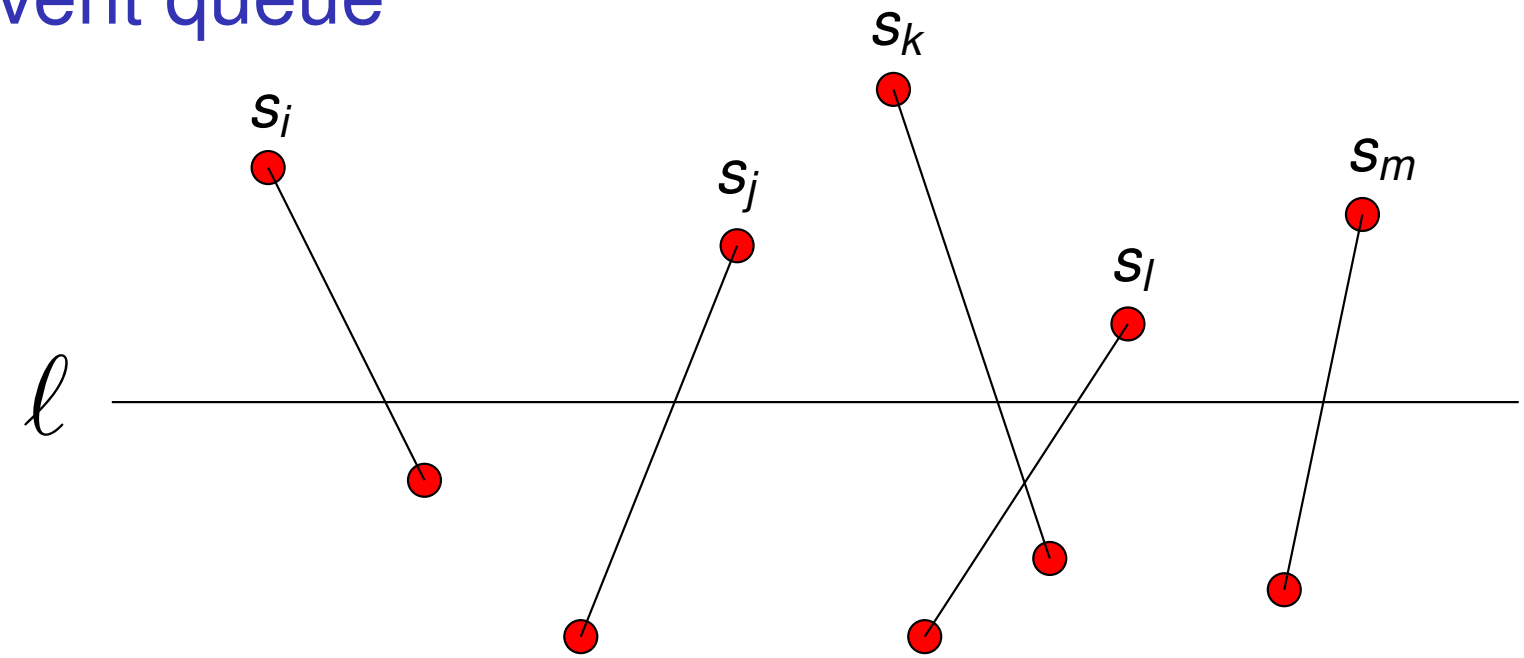
Algorithm FINDINTERSECTIONS(S)

Input: $S = \{ s_i((a_x^i, a_y^i), (b_x^i, b_y^i)), i = 1, 2, \ldots, n \}$.
Output: $P = \{ ((i,j),(x,y)) \mid (x,y) \in s_i \cap s_j \}$.

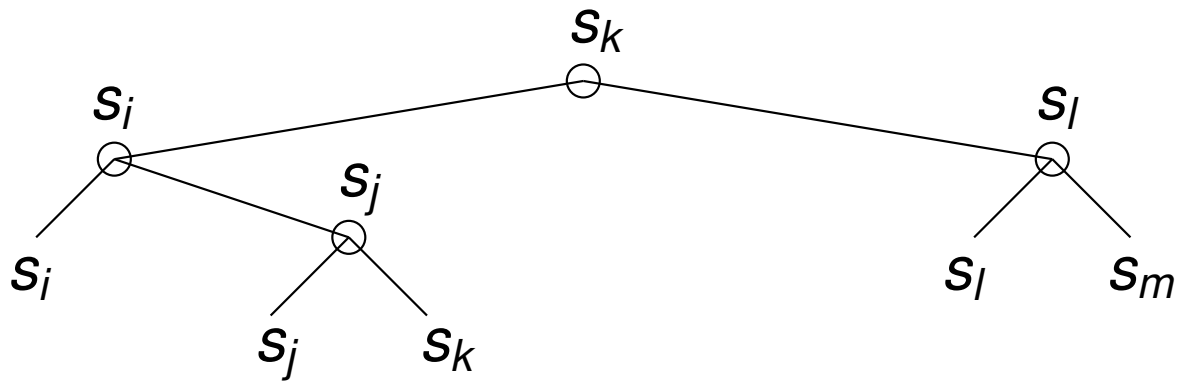
1. Initialize event queue Q with upper end points of the segments.
2. Initialize status $T = \emptyset$.
3. While $Q \neq \emptyset$ do
4. $\quad p =$ pop off next event point from Q,
5. $\quad$ HANDLEEVENTPOINT(p, Q, T).

Balanced binary search trees store the queue Q and status T.

an event queue

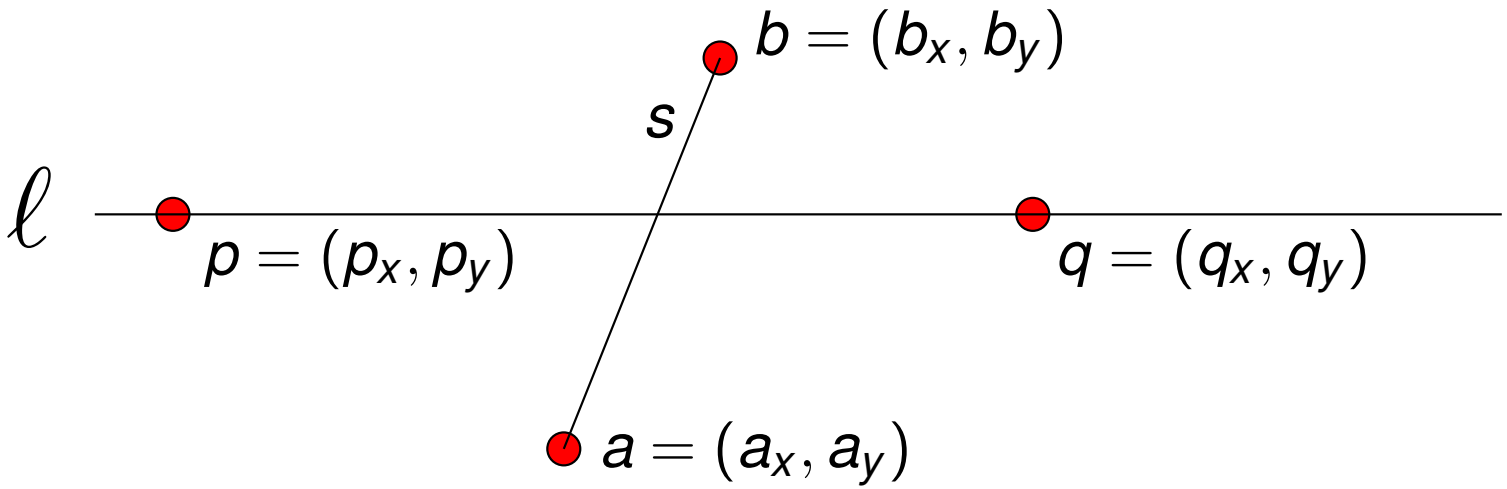


Initialization takes $O(n \log(n))$ time and $O(n)$ space.



deciding left or right

Exercise 1: Consider the segment $s = ((a_x, a_y), (b_x, b_y))$ and the points $p = (p_x, p_y)$ and $q = (q_x, q_y)$ as end points of new segments entering the status:



1. Given the coordinates of p, q, and the end points of s, how to decide that p lies to the left and q to the right of s?
2. Justify your criterion to decide if a point is at the left or at the right of a line segment.

Overlay of Maps

1. Output Sensitive Algorithms
   - the line segment intersection problem
   - handling event points
2. Correctness and Cost Analysis
   - correctness
   - cost analysis
3. Doubly Connected Edge Lists
   - representing planar subdivisions
   - vertex, face, and edge records

the sets $U(p)$, $L(p)$, $C(p)$

To define HANDLEEVENTPOINT(p, Q, T), we need the following sets:

- $U(p) = \{$ segments with upper end point $= p \}$ (incoming),
- $L(p) = \{$ segments in T with lower end point $= p \}$ (outgoing),
- $C(p) = \{$ segments in T which contain p in their interior $\}$.

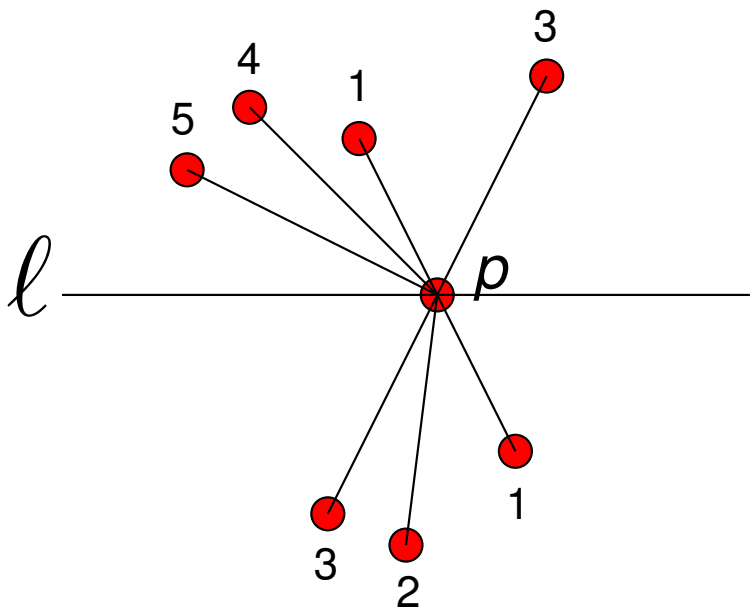
The subroutine HANDLEEVENTPOINT(p, Q, T) proceeds in 3 stages:

1. Compute the sets $U(p)$, $L(p)$, $C(p)$.
2. Update the status T.
3. Test for intersection.

the sets $U(p)$, $L(p)$, $C(p)$ – a (degenerate) example

To define HANDLEEVENTPOINT(p, Q, T), we need the following sets:

- $U(p) = \{$ segments with upper end point $= p \}$ (incoming),
- $L(p) = \{$ segments in T with lower end point $= p \}$ (outgoing),
- $C(p) = \{$ segments in T which contain p in their interior $\}$.



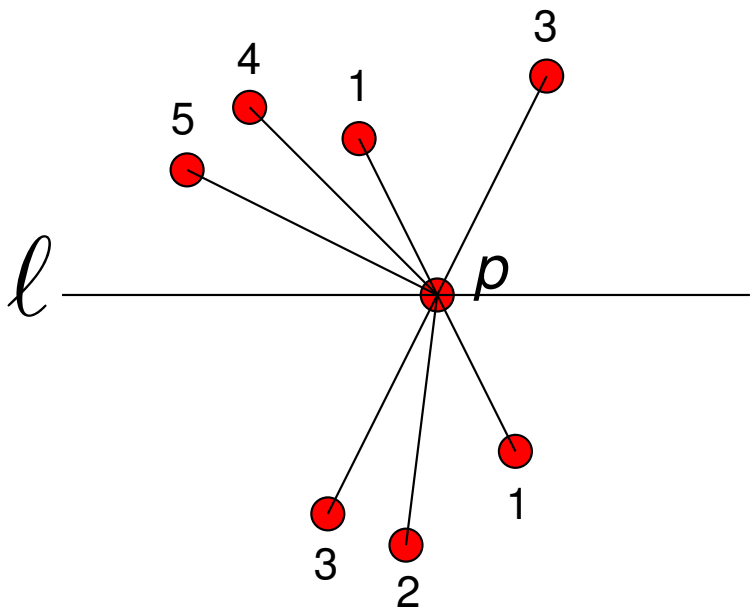
$U(p) = \{2\}$

$L(p) = \{4, 5\}$

$C(p) = \{1, 3\}$

Exercise 2: Consider the application of the algorithm to the (degenerate) example above. How many times is p computed?

the status before encountering p

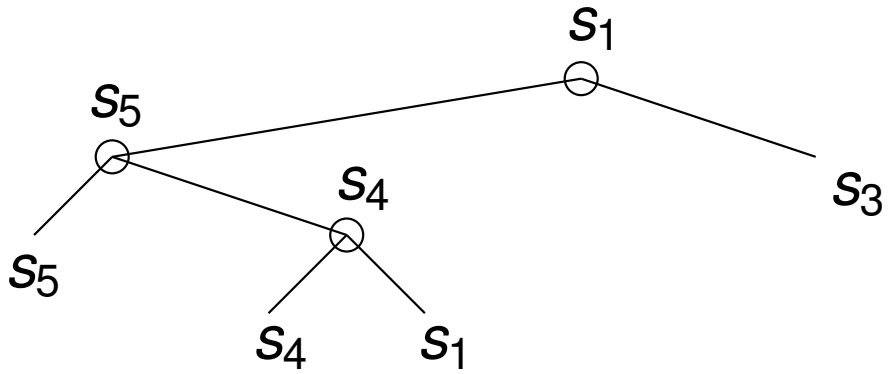


$U(p) = \{2\}$

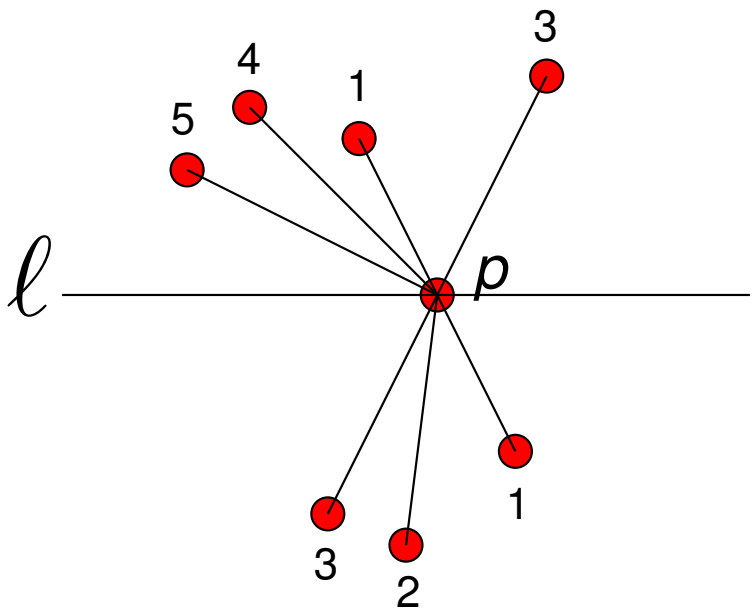
$L(p) = \{4, 5\}$

$C(p) = \{1, 3\}$

The status T:



the status after encountering p

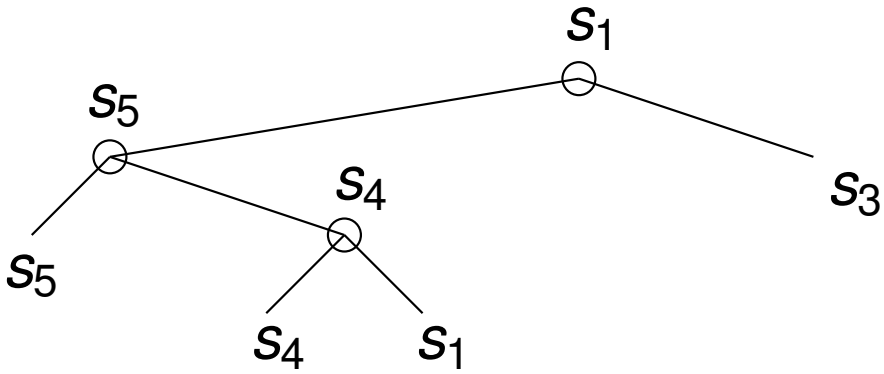


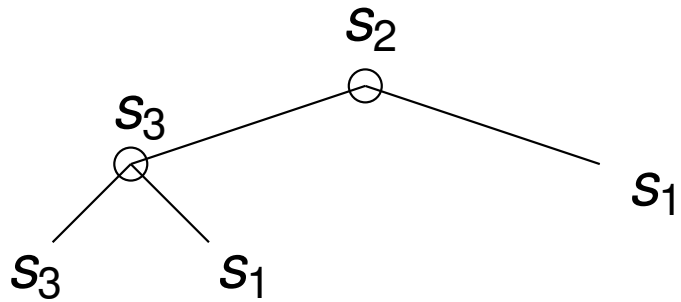
$U(p) = \{2\}$

$L(p) = \{4, 5\}$

$C(p) = \{1, 3\}$

The status T:





the algorithm to handle event points

Algorithm HANDLEEVENTPOINT(p, Q, T)

Input: p, an event point; Q, the event queue; T, the status.
Output: $P = \{ ((i,j),(x,y)) \mid (x,y) \in s_i \cap s_j \}$.

1. $U(p) = \{$ segments with upper end point $= p \}$
2. $L(p) = \{$ segments in T with lower end point $= p \}$
3. $C(p) = \{$ segments in T which contain p in their interior $\}$
4. if $\#(U(p) \cup L(p) \cup C(p)) > 1$ then
5. report p as intersection together with $U(p), L(p), C(p)$
6. delete segments $L(p) \cup C(p)$ from T
7. insert $U(p) \cup C(p)$ into T in the order in which they are intersected by the sweep line *just below* p

the algorithm continued

8. If $U(p) \cup C(p) = \emptyset$ then
9. $s_\ell =$ left neighbor of p in T; $s_r =$ right neighbor of p in T
10. FINDNEWEVENT(s_ℓ, s_r, p, Q)

else

11. $s' =$ leftmost segment of $U(p) \cup C(p)$ in T
12. $s_\ell =$ left neighbor of s' in T
13. FINDNEWEVENT(s_ℓ, s', p, Q)
14. $s'' =$ rightmost segment of $U(p) \cup C(p)$ in T
15. $s_r =$ right neighbor of s'' in T
16. FINDNEWEVENT(s'', s_r, p, Q)

the algorithm FINDNEWEVENT

Algorithm FINDNEWEVENT(s_ℓ, s_r, p, Q)

1. if s_ℓ and s_r intersect below the sweep line
2. or intersect on the sweep line at the right of p,
3. and if the intersection point is not yet in Q

then

4. insert the intersection point into Q

Overlay of Maps

1. Output Sensitive Algorithms
   - the line segment intersection problem
   - handling event points
2. Correctness and Cost Analysis
   - correctness
   - cost analysis
3. Doubly Connected Edge Lists
   - representing planar subdivisions
   - vertex, face, and edge records

two lemmas

In lecture 3, we showed the following lemma.

Lemma (detection of intersection points)

Let s_i and s_j be two line segments, both are not horizontal.
If $s_i \cap s_j = \{p\} \notin s_k$, $k \neq i$, $k \neq j$,
then there is an event point above ℓ where s_i and s_j are adjacent.

Lemma (correctness of FINDINTERSECTIONS)

Algorithm FINDINTERSECTIONS *computes all intersection points and the segments that contain them correctly.*

The priority of event points is determined by their y-coordinate.
The correctness lemma will be proven by induction on the priority.

proof of correctness

Base case: one segment, two event points, no intersection.

Induction hypothesis: at the current event point p, all intersection points with higher priority than p have been computed.

We need to show that p belongs to $U(p)$, $L(p)$, or $C(p)$.

- $U(p) = \{$ segments with upper end point $= p \}$ (incoming),
- $L(p) = \{$ segments in T with lower end point $= p \}$ (outgoing),
- $C(p) = \{$ segments in T which contain p in their interior $\}$.

If p is an end point: either $p \in U(p)$ or $p \in L(p)$,
in both cases, p is found and reported.

If p is not an end point, then
application of Lemma (detection of intersection points) shows that p is computed by the intersection test for two adjacent segments.

Q.E.D.

Overlay of Maps

1. Output Sensitive Algorithms
   - the line segment intersection problem
   - handling event points
2. Correctness and Cost Analysis
   - correctness
   - cost analysis
3. Doubly Connected Edge Lists
   - representing planar subdivisions
   - vertex, face, and edge records

FINDINTERSECTIONS is an output sensitive algorithm

Lemma (running time of FINDINTERSECTIONS)

For n segments, FINDINTERSECTIONS *runs in $O((n+I)\log(n))$ time, where I is the number of intersection points.*

Proof. The initialization of the event queue Q requires the sorting of the points on their y-coordinate, which takes $O(n\log(n))$.

As Q and the status T are stored by balanced binary search trees, each update of Q and T takes $O(\log(n))$ time.

Denote the number of steps by $m(p) = \#(L(p) \cup U(p) \cup C(p))$.
$m(p) = O(n+k)$, where k is proportional to the size of the output.

We need to show that $k = O(I)$.

a planar graph

Consider a graph G with vertices

- the end points of the line segments, and
- all intersection points.

Then the number of vertices $n_v = 2n + I$.

For an event point p, $m(p)$ is bounded by the degree of the vertex.
This degree is the number of edges incident to the vertex.
Thus, the number m of all event points $m \leq 2n_e$, where $n_e =$ #edges.

Denote $n_f =$ #faces. At least 3 edges bound one face: $n_f \leq 2n_e/3$.

Theorem (Euler's formula)

Let n_f, n_e, n_v respectively be the number of faces, the number of edges, and the number of vertices in a graph: $n_f - n_e + n_v \geq 2$.

Exercise 3: Verify Euler's formula for a triangle and a tree.
Sketch the outline of a proof by induction.

application of Euler's formula

By $m \leq 2n_e$, our quest to bound m (the total number of event points) is reduced to finding a bound on n_e.

We have

$$n_v = 2n + l, \quad n_f \leq 2n_e/3, \quad \text{and} \quad 2 \leq n_f - n_e + n_v.$$

$$\begin{array}{rcl} 2 & \leq & 2n_e/3 - n_e + (2n + l) \\ & \Downarrow & \\ 2 - 2n - l & \leq & -n_e/3 \\ & \Downarrow & \\ n_e & \leq & -6 + 6n + 3l \text{ is } O(n + l) \end{array}$$

Thus m is $O(n + l)$. Q.E.D.

storage cost

For n segments the storage cost could be $\Omega(n^2)$,
but by storing only data from adjacent segments,
the number of event ponts is never more than $O(n)$.

By the above observation and the Lemmas, we obtain:

Theorem (time and storage cost of FINDINTERSECTIONS)

Let S be a set of n line segments in the plane.
The set $P = \{ ((i,j),(x,y)) \mid (x,y) \in s_i \cap s_j \}$ can be computed in $O((n+I)\log(n))$ time, $I = \#P$, and $O(n)$ space.

Overlay of Maps

1. Output Sensitive Algorithms
   - the line segment intersection problem
   - handling event points
2. Correctness and Cost Analysis
   - correctness
   - cost analysis
3. Doubly Connected Edge Lists
   - representing planar subdivisions
   - vertex, face, and edge records

representing planar subdivisions

Find a data structure to represent a planar subdivision.

- a point is represented by a *vertex*,
- a line segment is an *edge* between two vertices,
- a polygonal region bounded by edges is a *face*.

The complexity is determined by (#vertices, #edges, #faces).

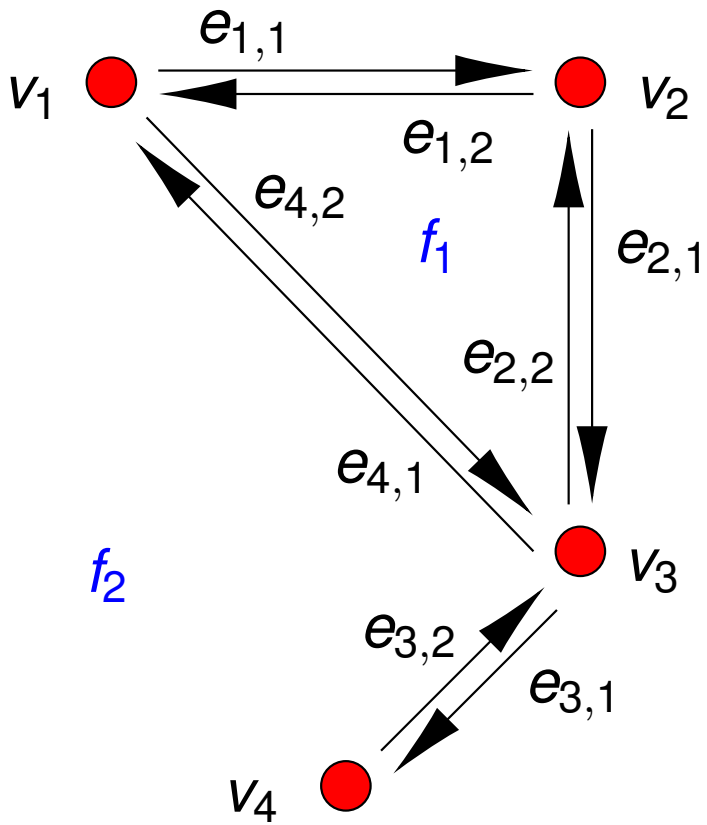
What can we ask from this data structure?

- Can we walk around the boundary of a given face?
- Can we access one face from an adjacent one?
- Can we visit all edges around a given vertex?

Definition (doubly connected edge list)

A *doubly connected edge list* contains a record for each face, edge, and vertex of the planar subdivision.
Additional information stored in a record is *attribute information*.

recording vertices, half edges, faces – an example



Overlay of Maps

1. Output Sensitive Algorithms
   - the line segment intersection problem
   - handling event points
2. Correctness and Cost Analysis
   - correctness
   - cost analysis
3. Doubly Connected Edge Lists
   - representing planar subdivisions
   - vertex, face, and edge records

a vertex record

Definition (vertex record)

A *vertex record* stores two items:

1. COORDINATES: (x, y) for the point, and
2. INCIDENTEDGE: the half edge e that originates at the vertex.

In the example on the previous slide, we have four vertices:

vertex	COORDINATES	INCIDENTEDGE
v_1	$(0, 4)$	$e_{1,1}$
v_2	$(2, 4)$	$e_{2,1}$
v_3	$(2, 2)$	$e_{3,1}$
v_4	$(1, 1)$	$e_{3,2}$

a face record

Definition (face record)

A *face record* stores two items:

1. INNERCOMPONENTS: half edges, one for each hole,
2. OUTERCOMPONENT: a half edge to its outer boundary.

In the example, we have two faces:

face	INNERCOMPONENTS	OUTERCOMPONENT
f_1	nil	$e_{2,2}$
f_2	$e_{1,1}$	nil

an edge record

Definition (edge record)

An *edge record* stores five items for the edge e:

1. ORIGIN: vertex where e starts,
2. TWIN: half edge in the opposite direction of e, the destination of e is ORIGIN(TWIN(e)),
3. NEXT: next half edge in the list, the destination of e is ORIGIN(NEXT(e)),
4. PREV: previous half edge in the list, the origin of e is ORIGIN(TWIN(PREV(e))),
5. INCIDENTFACE: face enclosed by e.

edge records for the example

EDGE	ORIGIN	TWIN	INCIDENTFACE	NEXT	PREV
$e_{1,1}$	v_1	$e_{1,2}$	f_2	$e_{2,1}$	$e_{4,1}$
$e_{2,1}$	v_2	$e_{2,2}$	f_2	$e_{3,1}$	$e_{1,1}$
$e_{3,1}$	v_3	$e_{3,2}$	f_2	$e_{3,2}$	$e_{2,1}$
$e_{3,2}$	v_4	$e_{3,1}$	f_2	$e_{3,1}$	$e_{4,1}$
$e_{4,1}$	v_3	$e_{4,2}$	f_2	$e_{3,2}$	$e_{1,1}$
$e_{4,2}$	v_1	$e_{4,1}$	f_1	$e_{2,2}$	$e_{1,2}$
$e_{2,2}$	v_3	$e_{2,1}$	f_1	$e_{1,2}$	$e_{4,2}$
$e_{1,2}$	v_2	$e_{1,1}$	f_1	$e_{4,2}$	$e_{2,2}$

an exercise

Exercise 4:

Let the triangle with coordinates $(-2, 0)$, $(0, 0)$, $(0, 2)$ be the hole in the quadrilateral with vertices $(-3, 0)$, $(3, 0)$, $(0, 3)$, $(3, 3)$.

1. Draw the planar subdivision with a labeling of all vertices, all half edges, and all faces.
2. Define all vertex records, edge records, and face records.

suggested activities

We continued the second chapter in the textbook.

Consider the following activities, listed below.

0. Browse the documentation for the half edge data structures:
   `https://doc.cgal.org/latest/HalfedgeDS/index.html`
1. Write the solutions to exercises 1, 2, 3, and 4.
2. Consider the exercises 4,5,6 in the textbook.